Discussion Module:

DSE Search, Query Syntax

Introduce DSE Search Query Syntax

Discussion Lab:

Tell Me Something I Don’t Know

Describe the Queries on the Pages That Follow-

Describe the Queries-

Core, Search
Field Type, Field Attributes
(Other objects)

CQL Predicates

```
WHERE col1 = 10;

WHERE col1 LIKE ‘Chuck’;

WHERE col1 = 10 AND col2 = 20;
WHERE col1 = 10 AND col2 = 20
   AND col3 > 30;

WHERE col1 = 10 and col2 = 20
ORDER BY col3;

WHERE solr_query =
  '{ "q" : "(name:D* OR name:DAVE~1^8)
  AND gender:M AND century:2000" }';
```

```
WHERE col3       > 'ddd';
WHERE col1 = 10 OR col2 = 20;

WHERE solr_query = '{ "q" : "col3:Dave" }' ;
    solr_query = '{ "q" : "col3:(Dava Rooney)" }' ;

    solr_query = '{ "q" : "col4:(zah hut)" }' ;
    solr_query = '{ "q" : "col4:Starrbux" }' ;

    solr_query = '{"q":"*:*", "sort":"col0  asc"}' LIMIT 12;

FROM t3 ORDER BY col6 ASC,
    col7 DESC, col3 ASC;

WHERE col6 > 12.0 AND col7 < 26;
WHERE col6 > 12.0 OR col7 < 26;
```

Describe the Queries-

Core, Search
Field Type, Field Attributes
(Other objects)

CQL Predicates

Default Search limit

Describe the Queries-

Core, Search
Field Type, Field Attributes
(Other objects)

CQL Predicates

```
WHERE solr_query = '{"q" : "col6:
   [12.0 TO *] AND col7:[* TO 26]",
"sort":"col6 ASC, col7 DESC, col3 ASC" }';

WHERE col5 LIKE  '%ar%';
WHERE col4 IN (2, 4);
WHERE col9 CONTAINS ('Vault');

WHERE solr_query = 'col8:
   [15.9 TO 16.00003]';
```

Describe the Queries-

```
WHERE solr_query = '{ "q" : "col73:
   (Chuck David Todd)"}';

WHERE solr_query = '{ "q" : "col73:
   (+Chuck +David +Todd)"}';

WHERE solr_query='col2:"test space"~10';

WHERE solr_query='"test space"~2';

WHERE solr_query = '{ "q" :
   "(col73:*huck^8 OR col74:chuck~1)" }' ;
```

Core, Search
Field Type, Field Attributes
(Other objects)

CQL Predicates

Default Search limit

End of Discussion Lab:

CQL SELECT: (Core, other)

SELECT ..
FROM ..
WHERE ..
GROUP BY ..
ORDER BY .. ;

(JOINS)

1.2.1

+

2.2.0.14

- LIMIT | PARTITION LIMIT
- UDFs, UDAs, (+ native funcs & aggs)
  (Core Yes, Core + Search, No)
- Paging (Not graph; Core and Search Yes)

DSE Search 6.0: Can not

A DSE Search query can only return column values from the DSE table proper:

- Not score, not any Solr virtual columns
- Not any Solr function queries
  (Returned to DSE No, used on Solr side Yes)
- No index time boost, only query time boost

https://lucene.apache.org/solr/guide/6_6/function-queries.html
https://datastax.jira.com/browse/DSP-8229
https://datastax.jira.com/browse/DSP-13788

DataStax Internal Use Only

DSE Search 6.0: Can not

A DSE Search query can not run DSE UDFs/UDAs-

```
CREATE OR REPLACE FUNCTION my_sum( i_arg1 int, i_arg2 int )
   RETURNS NULL ON NULL INPUT
   RETURNS int
   LANGUAGE JAVA AS '
   return i_arg1 + i_arg2 ;  ';

SELECT col1, my_sum(col3, col4) FROM t10
WHERE solr_query = '{ "q" : "col3 > 0" }';

// InvalidRequest: Error from server: code=2200 [Invalid query] message=
// "Aliased column names, UDFs and similar features are not available for
// Solr queries. Offending inputs are [col4] and [ks_7451.my_sum(col3, col4)]."
```

DataStax Internal Use Only

CQL SELECT: Search

solr_query = “”

- **CQL (non JSON)**
- **JSON**

Have already seen:

- **“q”**
- **Word + -**
- **Phrase (+ -)**
- **Boost ^**
- **Fuzzy ~**
- **sort**

```
{
  "q": query_expression (string),
  "fq": filter_query_expression(s) (string_or_array_of_strings, ...),
  "facet": facet_query_expression (object)
  "sort": sort_expression (string),
  "start": start_index(number),
 timeAllowed: search_time_limit_ms,
  "TZ": zoneID),   //  Any valid zone ID in java TimeZone class
  "paging": "driver" (string),
  "distrib.singlePass": true|false (boolean),
  "shards.failover": true|false (boolean),    // Default: true
  "shards.tolerant": true|false (boolean),    // Default: false
  "commit": true|false (boolean),
  "route.partition": partition_routing_expression (array_of_strings),
  "route.range": range_routing_expression (array_of_strings),
  "query.name": query_name (string),
}
```

Source: https://docs.datastax.com/en/dse/6.0/cql/cql/cql_using/search_index/siQuerySyntax.html#siQuerySyntax

Runtine used in the examples that follow

The next few pages detail the runtime environment used (tables, data, indexes) in the sample queries to follow-

Sample Runtime Used .. page 1 of 3


```
DROP KEYSPACE ks_7451;
CREATE KEYSPACE ks_7451 WITH
REPLICATION = {'class':
   'SimpleStrategy',
   'replication_factor': 1};
USE ks_7451;

CREATE TYPE my_udt
   (
   colm          TEXT,
   coln          TEXT
   );
```

```
CREATE TABLE t10
   (
   col1          TEXT,
   col2          TEXT,
   col3          INT,
   col4          INT,
   col5          TEXT,
   col6          TEXT,
   col7          TEXT,
   col8          TEXT,
   col9          SET <TEXT>,
   col0          FROZEN<my_udt>,
   PRIMARY KEY ((col1, col2)) );
```

Sample Runtime Used .. page 2 of 3


```
INSERT INTO t10
   (col1, col2, col3, col4, col5,
      col6, col7, col8, col9, col0)
VALUES
   ('aaa', 'aaa', 1, 2, 'Mary',
      'Dog Cat Bird', 'aaa', 'aaa',
      {'Coke', 'Diet Coke'},
      { colm : 'aaa' , coln : 'aaa' } );

VALUES
   ('bbb', 'bbb', 3, 4, 'Harry',
      'Dog Mouse Cat', 'bbb', 'bbb',
      {'Mtn Dew', 'Vault'},
      { colm : 'bbb' , coln : 'bbb' } );
```

```
VALUES
   ('ccc', 'ccc', 1, 5, 'Dave',
      'Dog Mule Cat', 'ccc', 'ccc',
      {'Sprite', '7-Up'},
      { colm : 'ccc' , coln : 'ccc' } );

VALUES
   ('ddd', 'ddd', 1, 6, 'David',
      'Cat Dog Bird', 'ddd', 'ddd',
      {'Orange'},
      { colm : 'ddd' , coln : 'ddd' } );
```

Sample Runtime Used .. page 3 of 3


```
CREATE SEARCH INDEX ON t10 WITH
COLUMNS col1, col2, col3,
   col4, col9, col0 { docValues : true };

ALTER SEARCH INDEX SCHEMA ON t10
    ADD types.fieldType[@name='TextField_10',
    @class='solr.TextField']
    with '{"analyzer":{"tokenizer":
    {"class":"solr.StandardTokenizerFactory"} }}';

ALTER SEARCH INDEX SCHEMA ON t10
    ADD fields.field[@name='col5',
    @type='TextField_10', @indexed='true',
    @multiValued='false', @stored='true'];
```

```
ALTER SEARCH INDEX SCHEMA ON t10
    ADD fields.field[@name='col6',
    @type='TextField_10', @indexed='true',
    @multiValued='false', @stored='true'];

RELOAD  SEARCH INDEX ON t10;
REBUILD SEARCH INDEX ON t10;
```

What did each column receive index wise ?

DSE Search CQL SELECT Syntax

Source: https://www.autoblog.com/2018/02/12/range-rover-phev-china-heavens-gate/

Query: Term

SELECT col1, col5 FROM t10 WHERE
solr_query = '{ "q" : "col5:Mary" }';

```
 col1 | col5
------+------
  aaa | Mary
```

solr_query = '{ "q" : "col5:(Mary || Harry)" }';

```
 col1 | col5
------+-------
  bbb | Harry
  aaa |  Mary
```

Data is :

```
'aaa', 'Mary' , 'Dog Cat Bird'
'bbb', 'Harry', 'Dog Mouse Cat'
'ccc', 'Dave' , 'Dog Mule Cat'
'ddd', 'David', 'Cat Dog Bird'
```

Query: Term

SELECT col1, col6 FROM t10 WHERE solr_query = '{ "q" : "col6:Dog" }';

```
 col1 | col6
------+---------------
  ddd |  Cat Dog Bird
  ccc |  Dog Mule Cat
  bbb | Dog Mouse Cat
  aaa |  Dog Cat Bird
```

Data is :

```
'aaa', 'Mary' , 'Dog Cat Bird'
'bbb', 'Harry', 'Dog Mouse Cat'
'ccc', 'Dave' , 'Dog Mule Cat'
'ddd', 'David', 'Cat Dog Bird'
```

Query: Phrase

```
SELECT col1, col6 FROM t10
WHERE solr_query = '{ "q" :
   "col6:(Dog Mule Cat)" }';

 col1 | col6
------+---------------
  ccc |  Dog Mule Cat
  ddd |  Cat Dog Bird
  bbb | Dog Mouse Cat
  aaa |  Dog Cat Bird
```

Boolean ?

Data is :

```
'aaa', 'Mary' , 'Dog Cat Bird'
'bbb', 'Harry', 'Dog Mouse Cat'
'ccc', 'Dave' , 'Dog Mule Cat'
'ddd', 'David', 'Cat Dog Bird'
```

```
WHERE solr_query = '{ "q" :
   "col6:(+Dog +Mule +Cat)" }';

 col1 | col6
------+-------------
  ccc | Dog Mule Cat
```

Boolean ?

Query: Phrase

```
SELECT col1, col6 FROM t10
WHERE solr_query = '{ "q" :
"col6:(+Dog +Cat && -Mouse -Mule)" }';
```

```
 col1 | col6
------+--------------
  ddd | Cat Dog Bird
  aaa | Dog Cat Bird
```

```
WHERE solr_query = '{ "q" :
  "col6:(+Dog -Mule +Cat)" }';
```

```
 col1 | col6
------+---------------
  ddd |  Cat Dog Bird
  bbb | Dog Mouse Cat
  aaa |  Dog Cat Bird
```

Data is :

```
'aaa', 'Mary' , 'Dog Cat Bird'
'bbb', 'Harry', 'Dog Mouse Cat'
'ccc', 'Dave' , 'Dog Mule Cat'
'ddd', 'David', 'Cat Dog Bird'
```

Query: Range

```
SELECT col1, col4 FROM t10
WHERE solr_query = '{ "q" :
   "col4:[4 TO 5]" }';
```

```
 col1 | col4
------+------
  ccc |    5
  bbb |    4
```

```
SELECT col1, col6 FROM t10
WHERE solr_query = '{ "q" :
   "col6:[Mouse TO Mule]" }';
```

```
 col1 | col6
------+---------------
  ccc |  Dog Mule Cat
  bbb | Dog Mouse Cat
```

Data is :

```
  'aaa', 'Mary' , 'Dog Cat Bird'
  'bbb', 'Harry', 'Dog Mouse Cat'
  'ccc', 'Dave' , 'Dog Mule Cat'
  'ddd', 'David', 'Cat Dog Bird'
```

Query: Substring

SELECT col1, col5 FROM t10 WHERE
solr_query = '{ "q" : "col5:Mar*" }';

solr_query = '{ "q" : "col5:*ary" }';

```
 col1 | col5
------+------
  aaa | Mary
```

(Both)

solr_query = '{ "q" : "col5:*ar*" }';

```
 col1 | col5
------+-------
  bbb | Harry
  aaa |  Mary
```

Data is :

```
'aaa', 'Mary' , 'Dog Cat Bird'
'bbb', 'Harry', 'Dog Mouse Cat'
'ccc', 'Dave' , 'Dog Mule Cat'
'ddd', 'David', 'Cat Dog Bird'
```

Performance implication ?

Query: Substring, and negate

SELECT col1, col6 FROM t10 WHERE
solr_query = '{ "q" : "col6:
(+Dog +Cat && -M*)" }';

```
 col1 | col6
------+--------------
  ddd | Cat Dog Bird
  aaa | Dog Cat Bird
```

Data is :

```
'aaa', 'Mary' , 'Dog Cat Bird'
'bbb', 'Harry', 'Dog Mouse Cat'
'ccc', 'Dave' , 'Dog Mule Cat'
'ddd', 'David', 'Cat Dog Bird'
```

Query: Fuzzy Search

```
SELECT col1, col5 FROM t10
WHERE solr_query = '{ "q" :
  "col5:arry~2" }';
```

```
 col1 | col5
------+-------
  bbb | Harry
  aaa |  Mary
```

Data is :

```
  'aaa', 'Mary' , 'Dog Cat Bird'
  'bbb', 'Harry', 'Dog Mouse Cat'
  'ccc', 'Dave' , 'Dog Mule Cat'
  'ddd', 'David', 'Cat Dog Bird'
```

Query: Word Proximity

SELECT col1, col6 FROM t10 WHERE
solr_query = 'col6:"Cat Bird"~0';

```
 col1 | col6
------+--------------
  aaa | Dog Cat Bird
```

solr_query = 'col6:"Cat Bird"~1;

```
 col1 | col6
------+--------------
  aaa | Dog Cat Bird
  ddd | Cat Dog Bird
```

Data is :

```
'aaa', 'Mary' , 'Dog Cat Bird'
'bbb', 'Harry', 'Dog Mouse Cat'
'ccc', 'Dave' , 'Dog Mule Cat'
'ddd', 'David', 'Cat Dog Bird'
```

Query: Boosting

SELECT col1, col6 FROM t10 WHERE solr_query = '{ "q" : "(col6:Mule OR col6:Mouse^16)" }';

```
 col1 | col6
------+---------------
  bbb | Dog Mouse Cat
  ccc |  Dog Mule Cat
```

SELECT col1, col6 FROM t10 WHERE solr_query = '{ "q" : "(col6:Mule OR col6:Mouse^=1.0)" }' LIMIT 1 ;

```
 col1 | col6
------+---------------
  bbb | Dog Mouse Cat
```

Data is :

```
'aaa', 'Mary' , 'Dog Cat Bird'
'bbb', 'Harry', 'Dog Mouse Cat'
'ccc', 'Dave' , 'Dog Mule Cat'
'ddd', 'David', 'Cat Dog Bird'
```

Query: Joins

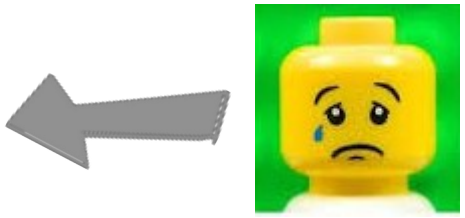
Apache Solr joins do not equal SQL joins;
most similar to a nested SELECT;
https://wiki.apache.org/solr/Join

In effect, this Apache Solr search statement,

```
WHERE solr_query='{"q":"*:*", "fq":
   "{!join from=cola to=col5 force=true
   fromIndex=ks_7451.t11} colb:Bob"}';
```

Would equal the legacy SQL SELECT statement,

```
SELECT ...
WHERE outer_id IN (SELECT inner_id
   FROM collection1 where cola = col5
   AND colb = “Bob”;
```



Query: Joins, how to

```
DROP TABLE t11;
CREATE TABLE t11
   (
   cola              TEXT,
   colb              TEXT,
   PRIMARY KEY ((cola))
   );
INSERT INTO t11 (cola, colb)
    VALUES ('Mary', 'Schulte');

    VALUES ('Harry', 'Johnson');
    VALUES ('Dave', 'Wilson');
    VALUES ('David', 'Bowie');

CREATE SEARCH INDEX ON t11;
```

Query: Joins, how to

```
SELECT col1, col5 FROM t10
WHERE solr_query='{"q":"*:*", "fq":
   "{!join from=cola to=col5 force=true
   fromIndex=ks_7451.t11} colb:Schulte"}';
```

```
 col1 | col5
------+------
  aaa | Mary
```

```
WHERE solr_query='{"q":"*:*", "fq":
   "{!join from=cola to=col5 force=true
   fromIndex=ks_7451.t11}*:*"}';
```

```
 col1 | col5
------+-------
  ddd | David
  ccc |  Dave
  bbb | Harry
  aaa |  Mary
```

Data is :

```
'aaa', 'Mary' , 'Dog Cat Bird'
'bbb', 'Harry', 'Dog Mouse Cat'
'ccc', 'Dave' , 'Dog Mule Cat'
'ddd', 'David', 'Cat Dog Bird‘

'Mary', 'Schulte'
'Harry', 'Johnson'
'Dave', 'Wilson'
'David', 'Bowie'
```

Query: Collections (Set, List, Map)

```
CREATE TABLE ...
  col9            SET <TEXT>
```

```
SELECT col1, col9 FROM t10 WHERE
solr_query='col9:"Mtn Dew"';
```

```
 col1 | col9
------+----------------------
  bbb | {'Mtn Dew', 'Vault'}
```

Query: UDTs

SELECT col1, col0 FROM t10
WHERE solr_query='{!tuple}col0.colm:ddd';

```
 col1 | col0
------+-----------------------------
  ddd | {colm: 'ddd', coln: 'ddd'}
```

WHERE solr_query='-{!tuple}col0.colm:ddd';

```
 col1 | col0
------+-----------------------------
  ccc | {colm: 'ccc', coln: 'ccc'}
  bbb | {colm: 'bbb', coln: 'bbb'}
  aaa | {colm: 'aaa', coln: 'aaa'}
```

Query: Solr Server Side “Function Queries”

```
SELECT col1, col3, col4 FROM t10
WHERE
solr_query = '{ "q":"*:*",
   "sort":"sum(col3, col4)  asc" }';
```

```
 col1 | col3 | col4
------+------+------
  aaa |    1 |    2
  ccc |    1 |    5
  ddd |    1 |    6
  bbb |    3 |    4
```

- See ,https://lucene.apache.org/solr/guide/6_6/function-queries.html
- 20 or more by count, not all are relevant

- Used in predicates, value can not be returned to the client

Query: DSE UDF/UDA and Search

cassandra.yaml
```
enable_user_defined_functions=true
```

```
CREATE OR REPLACE FUNCTION
    my_sum( i_arg1 int, i_arg2 int )
  RETURNS NULL ON NULL INPUT
  RETURNS int
  LANGUAGE JAVA AS '
    return i_arg1 + i_arg2 ;
  ';
```

```
SELECT col1, my_sum(col3, col4)
FROM t10
WHERE solr_query = '{ "q" : "col3 > 0" }';
```

InvalidRequest: Error from server: code=2200 [Invalid query] message= "Aliased column names, UDFs and similar features are not available for Solr queries.

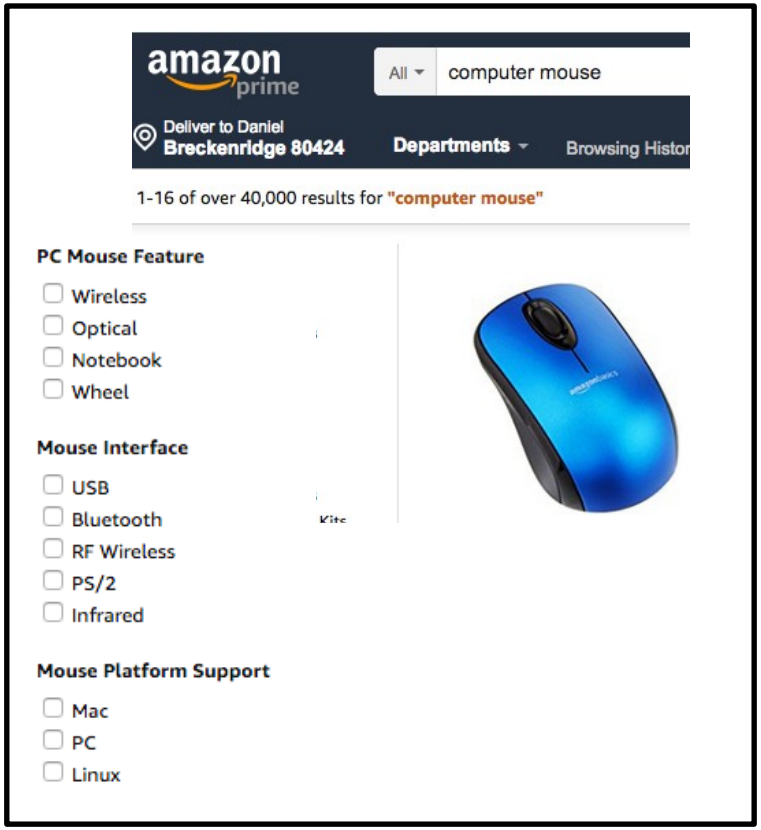
Query: Regex

SELECT col1, col6 FROM t10 WHERE solr_query = '{ "q" : "col6:/[Dd]og/" }';

```
 col1 | col6
------+---------------
  ddd |  Cat Dog Bird
  ccc |  Dog Mule Cat
  bbb | Dog Mouse Cat
  aaa |  Dog Cat Bird
```

Also available as a Filter; PatternReplaceFilter

DSE Search, Query: Facets



Query: Facets, “facet”

“facet”

```
SELECT * FROM t10 WHERE
solr_query='{"q":"*:*",
   "facet":{"field":"col3"}}';

 facet_fields
-------------------------
 {"col3":{"1":3,"3":1}}
```

Data is :

```
  'aaa', 'aaa', 1, 2, 'aaa',
  'bbb', 'bbb', 3, 4, 'bbb',
  'ccc', 'ccc', 1, 5, 'ccc',
  'ddd', 'ddd', 1, 6, 'ddd',
```

Query: Facets, “query facet”

“query facet”

```
SELECT * FROM t10
WHERE solr_query='{"q":"*:*",
   "facet":{"query":"col3:1"}}';
```

```
 facet_queries
---------------
  {"col3:1":3}
```

```
solr_query='{"q":"col3:1",
   "facet":{"field":"col3"}}';
```

```
 facet_fields
------------------------
 {"col3":{"1":3,"3":0}}
```

Data is :

```
  'aaa', 'aaa', 1, 2, 'aaa',
  'bbb', 'bbb', 3, 4, 'bbb',
  'ccc', 'ccc', 1, 5, 'ccc',
  'ddd', 'ddd', 1, 6, 'ddd',
```

Query, Facets, “multiple query”

```
SELECT * FROM t10
WHERE solr_query='{"q":"*:*",
   "facet":{"query":["col3:[0 TO *]",
   "col4:[* TO 5]"]}}';
```

```
 facet_queries
---------------------------------------
 {"col3:[0 TO *]":4,"col4:[* TO 5]":3}
```

Data is :

```
   'aaa', 'aaa', 1, 2, 'aaa',
   'bbb', 'bbb', 3, 4, 'bbb',
   'ccc', 'ccc', 1, 5, 'ccc',
   'ddd', 'ddd', 1, 6, 'ddd',
```

Query: Facet, “distributed pivot facet”, aka, decision tree”

```
SELECT col1 FROM t10
WHERE solr_query='{"q":"col1:*",
   "facet":{"pivot":"col3,col4",
   "limit":"-1"}}';
```

See formatted query result on notes page.

```
"col3,col4" :
    "field" : "col3",
    "value" : 1,
    "count" : 3,
    "pivot" :
       "field" : "col4",
       "value" : 2,
       "count" : 1
       "field" : "col4",
       "value" : 5,
       "count" : 1
       "field" : "col4",
       "value" : 6,
       "count" : 1
    "field" : "col3",
    "value" : 3,
    "count" :1,
    "pivot" :
       "field" : "col4",
       "value" :4,
       "count" : 1
```

PC Mouse Feature

- Wireless
- Optical
- Notebook
- Wheel

Mouse Interface

- USB
- Bluetooth
- RF Wireless
- PS/2
- Infrared

Mouse Platform Support

- Mac
- PC
- Linux

Query: Facets, range facet, aka “bucketing”

```
SELECT * FROM t10
WHERE solr_query=
'{"q":"col1:*",
   "facet":{"range":"col3",
"f.col3.range.start":-10,
"f.col3.range.end":10,
   "range.gap":2} }';
```

```
"col3" :
   "counts" :
      "-10" : 0,
      "-8"  : 0,
      "-6"  : 0,
      "-4"  : 0,
      "-2"  : 0,
      "0"   : 3,
      "2"   : 1,
      "4"   : 0,
      "6"   : 0,
      "8"   : 0
   "gap"    : 2,
   "start"  : -10,
   "end"    : 10
```

DSE Search, Query:

(switches), query parsers, “q” / “fq”

Query: (switches)

```
SELECT * FROM t10
WHERE solr_query = '{"q" : "*:*",
"distrib.singlePass" : true}';

//  Value is in ms, this example 30 secs
SELECT * FROM t10
WHERE solr_query = '{"q" : "*:*",
"timeAllowed":30000}';
```

- *Directing a Search query to a specific node; see Url*
- Other

Single pass (true):

- More disk, network total
- Saves 1 network round trip

- Generally, less efficient, more expensive

- Why ? Huge network latency

Query: Parsers

Default is: Standard Query Parser
https://lucene.apache.org/solr/guide/6_6/the-standard-query-parser.html

Many others (Urls on notes page):
- dismax
- edismax (Extended dismax)
- (others)

Why:
- Change default Boolean behavior
- Better/easier support for phrases
- Need new predicates-
- *10+ other reasons*

```
SELECT col1, col6 FROM t10
WHERE solr_query='{"q" :
   "{!edismax qf=col6}Dog AND Cat"}';
```

```
 col1 | col6
------+--------------
  ddd |  Cat Dog Bird
  ccc |  Dog Mule Cat
  bbb | Dog Mouse Cat
  aaa |  Dog Cat Bird
```

Query: “q” and “fq” (query, filter query)

- Most queries thus far (JSON) have been “q”, query predicate
- “fq” are a further set of predicates applied to the results of “q”
  - Not a correlated subquery or similar, (no new columns, rows)
  - Just further (filtering)

- fq filter activity does not affect score
- Most common means to OOM: DSE Tech Support, *use wisely*

“This parameter can be used to specify a query that can be used to restrict the super set of documents that can be returned, without influencing score.

It can be very useful for speeding up complex queries since the queries specified with fq are cached independently from the main query.

Caching means the same filter is used again for a later query.

Source: https://wiki.apache.org/solr/CommonQueryParameters#fq

Query: “q” and “fq” (query, filter query)

```
SELECT col1, col6 FROM t10
WHERE solr_query = '{ "q" :"col6:Dog",
   "fq" : "col6:Cat", "fq" : "col6:Bird" } ';
```

```
 col1 | col6
------+--------------
  ddd | Cat Dog Bird
  aaa | Dog Cat Bird
```

DSE Search CQL SELECT Syntax

Review

End of Module:

Standard Query Parser: Parameters

- “q”, the standard, required parameter we have seen throughout
- “df”, specify a query time default field
- “q.op”, change the default OR condition to AND
- Begin with “{!”, end with “}”, any number of key value pairs

Standard Query Parser: Parameters, df

```
SELECT col1, col6 FROM t10
WHERE solr_query =
   '{ "q" : "col6:(Mouse Mule)" }';

   '{ "q" : "{! df=col6}(Mouse Mule)" }';
```

```
 col1 | col6
------+---------------
  ccc |  Dog Mule Cat
  bbb | Dog Mouse Cat
```

Data is :

```
  'aaa', 'Mary' , 'Dog Cat Bird'
  'bbb', 'Harry', 'Dog Mouse Cat'
  'ccc', 'Dave' , 'Dog Mule Cat'
  'ddd', 'David', 'Cat Dog Bird'
```

Standard Query Parser: Parameters, q.op

```
SELECT col1, col6 FROM t10
WHERE solr_query = '{ "q" :
   "{! q.op=AND}col6:(Mouse Mule)" }';

 col1 | col6
------+------
(0 rows)
```

...

```
'{ "q" : "{! q.op=OR}col6:(Mouse Mule)" }';

 col1 | col6
------+----------------
  ccc |  Dog Mule Cat
  bbb | Dog Mouse Cat
```

Data is :

```
'aaa', 'Mary' , 'Dog Cat Bird'
'bbb', 'Harry', 'Dog Mouse Cat'
'ccc', 'Dave' , 'Dog Mule Cat'
'ddd', 'David', 'Cat Dog Bird'
```

dismax Query Parser: Parameters

- “q”, the standard
- “q.alt”, Calls the standard query parser and defines query input strings, when the q parameter is not used.
- “qf”, Query Fields: specifies the fields in the index on which to perform the query. If absent, defaults to df.
- “mm”, Minimum "Should" Match: specifies a minimum number of clauses that must match in a query.
- “pf”, Phrase Fields: boosts the score of documents in cases where all of the terms in the q parameter appear in close proximity.
- “ps”, Phrase Slop: specifies the number of positions two terms can be apart in order to match the specified phrase.

dismax Query Parser: Parameters

- “qs”, Query Phrase Slop: specifies the number of positions two terms can be apart in order to match the specified phrase. Used specifically with the qf parameter.
- “tie”, Tie Breaker: specifies a float value (which should be something much less than 1) to use as tiebreaker in DisMax queries. Default: 0.0
- “bq”, Boost Query: specifies a factor by which a term or phrase should be "boosted" in importance when considering a match.
- “bf”, Boost Functions: specifies functions to be applied to boosts. (See for details about function queries.)

edismax Query Parser: Parameters

- “sow”, Split on whitespace: if set to false, whitespace-separated term sequences will be provided to text analysis in one shot, enabling proper function of analysis filters that operate over term sequences, e.g. multi-word synonyms and shingles. Defaults to true: text analysis is invoked separately for each individual whitespace-separated term.
- “mm.autoRelax”, If true, the number of clauses required (minimum should match) will automatically be relaxed if a clause is removed (by e.g. stopwords filter) from some but not all qf fields.
- “boost”, A multivalued list of strings parsed as queries with scores multiplied by the score from the main query for all matching documents.
- “lowercaseOperators”, A Boolean parameter indicating if lowercase "and" and "or" should be treated the same as operators "AND" and "OR".

edismax Query Parser: Parameters

- “ps”, Default amount of slop on phrase queries built with pf, pf2 and/or pf3 fields (affects boosting).
- “pf2”, A multivalued list of fields with optional weights, based on pairs of word shingles.
- “ps2”, This is similar to ps but overrides the slop factor used for pf2. If not specified, ps is used.
- “pf3”, A multivalued list of fields with optional weights, based on triplets of word shingles. Similar to pf, except that instead of building a phrase per field out of all the words in the input, it builds a set of phrases for each field out of each triplet of word shingles.

edismax Query Parser: Parameters

- “ps3”, This is similar to ps but overrides the slop factor used for pf3. If not specified, ps is used.
- “stopwords”, A Boolean parameter indicating if the StopFilterFactory configured in the query analyzer should be respected when parsing the query: if it is false, then the StopFilterFactory in the query analyzer is ignored.
- “uf”, Specifies which schema fields the end user is allowed to explicitly query. This parameter supports wildcards. The default is to allow all fields, equivalent to uf=*. To allow only title field, use uf=title. To allow title and all fields ending with '_s', use uf=title,*_s. To allow all fields except title, use uf=*,-title. To disallow all fielded searches, use uf=-*.
- “qf”, Per-field overrides of the qf parameter may be specified to provide 1-to-many aliasing from field names specified in the query string, to field names used in the underlying query. By default, no aliasing is used and field names specified in the query string are treated as literal field names in the index.

edismax Query Parser: Parameters

- “_val_”, If the magic field name _val_ is used in a term or phrase query, the value is parsed as a function.It provides a hook into FunctionQuery syntax. Quotes are necessary to encapsulate the function when it includes parentheses.

  For example:_val_:myfield_val_:"recip(rord(myfield),1,2,3)“

- “_query_”, The Solr Query Parser offers nested query support for any type of query parser (via QParserPlugin). Quotes are often necessary to encapsulate the nested query if it contains reserved characters.

  For example:_query_:"{!dismax qf=myfield}how now brown cow"

Paging-

- Defined as; not for batch or related, but for end user activity, return a large set of rows in manageable chunks

- Basic pagination, use Solr “start” and “rows” parameters. Use queryResultCache, and queryResultWindowSize parameters.
- The above can be disabled in dse.yaml when cql_solr_query_paging:off.

- Cursor based pagination, use the “paging:driver” query parameter. Example-

  solr_query = {“q”:””, “sort”:”col1 asc”, “paging”:”driver”}’;

QueryResponseWriter

- Only applicable to Http queries. (The same used by SolrJ, the Solr Admin UI, other.)
- BinaryResponseWriter is used internally by CQL for serialization/deserialization of SolrQueryResponse.
- CQL operates on rows, or JSON

SELECT json col1, col5 FROM t10 WHERE

solr_query = '{ "q" : "col5:(Mary || Harry)" }';

```
[json]
----------------------------------
 {"col1": "bbb", "col5": "Harry"}
  {"col1": "aaa", "col5": "Mary"}
```

DataStax Internal Use Only